Relatório de Auditoria de Segurança

MegaSena

1 de setembro de 2026

Escopo auditado	Repositório MegaSena na íntegra: as 22 rotas HTTP de `app/web/`, as camadas de serviço (`app/bets`, `app/draws`, `app/settings`, `app/accounts`, `app/audit`), modelos e migrações, os 22 templates Jinja, o JavaScript de `app/static/`, `Dockerfile`, `compose.yaml`, `docker-entrypoint.sh`, `deploy/nginx/`, o workflow de CI, os arquivos de ambiente (versionados e não versionados) e o histórico completo do Git.
Linguagem	Python 3.14
Framework	Flask (monólito modular: `app/web` HTTP sobre casos de uso)
ORM / query builder	SQLAlchemy + Flask-SQLAlchemy; migrações por Alembic/Flask-Migrate
Banco	PostgreSQL 17 (único backend suportado; a recusa a outro dialeto é explícita no boot)
Mecanismo de auth	SharedAuth v0.10 — sessão de cookie assinado + Flask-Login, CSRF por Flask-WTF, portão padrão-nega por `before_request`, papéis `admin`/`operador` decididos localmente
Frontend	HTML renderizado no servidor + HTMX 2.0.10 (vendorizado); sem framework de componentes
Deploy	Docker Compose (imagem `runtime` com gunicorn e usuário não-root), Nginx com TLS no VPS Oracle, GitHub Actions com pip-audit e Trivy

Nota metodológica

As cinco categorias foram traduzidas para esta stack antes de qualquer busca. Duas delas — isolamento de inquilino e IDOR — dependem de o sistema ter noção de dono do dado, e este NÃO tem: o AGENTS.md declara que 'o sistema autentica, mas não particiona dados; qualquer conta autenticada acessa o acervo comum'. A auditoria confirmou isso no schema (nenhuma tabela de domínio tem coluna de dono) e percorreu assim mesmo TODOS os 22 handlers de rota, um a um, para separar o que é decisão consciente do que seria descuido. O resultado está na seção 2.3.

Categoria pedida	Como foi mapeada nesta stack
1. Banco sem tranca (isolamento de inquilino/dono)	Não há RLS nem middleware de tenant, e não há a que aplicá-los: `Draw`, `GeneratedBet` e `Config` não têm coluna de dono (verificado em `app/models.py` e nas 5 revisões Alembic). O mecanismo de isolamento efetivo é o portão padrão-nega de sessão mais o papel `admin` em 8 rotas. Auditadas todas as consultas de listagem, busca, estatística e importação.

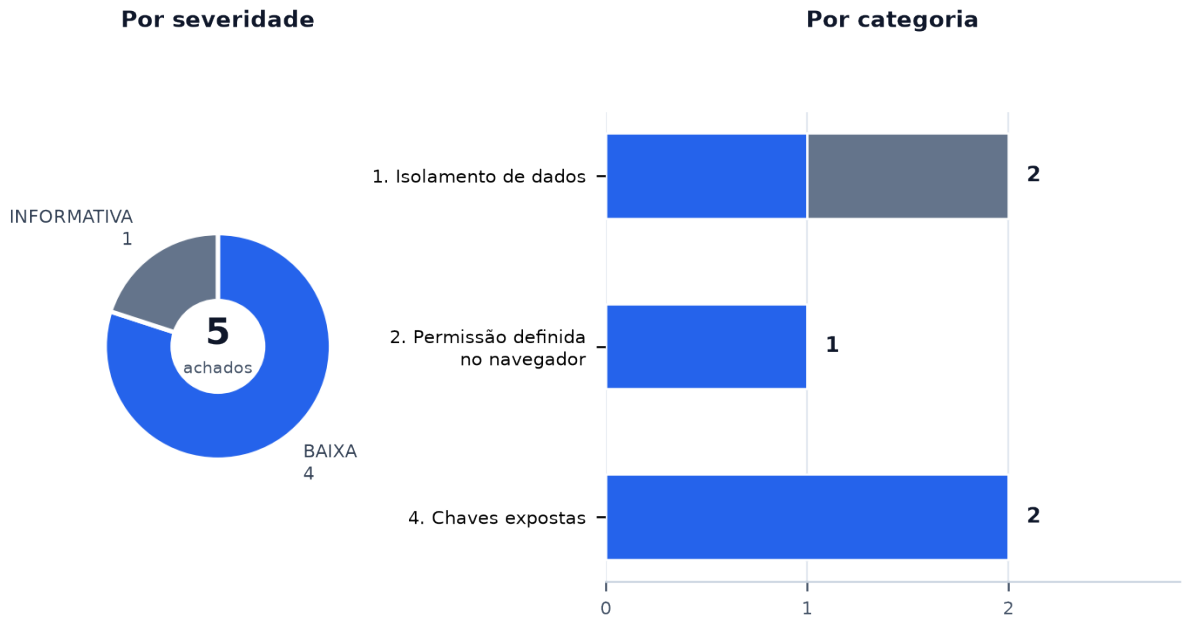
Categoria pedida	Como foi mapeada nesta stack
2. Permissão definida no navegador	Cruzamento de cada gate de papel do template (`base.html`) com o decorator da rota correspondente, e varredura das 22 rotas registradas para checar quais têm `@admin_required`.
3. IDOR	Percorridos os 4 handlers que recebem ID no caminho (`/usuarios/<id>/senha`, `/ativo`, `/papel` e `/bets/saved/<id>/delete`) e os que recebem ID em query ou corpo. Como não há dono, a pergunta auditada foi: a rota confere o PRIVILÉGIO de quem chama e a consistência do objeto?
4. Chaves expostas (hardcode)	Varredura de código, templates, `compose.yaml`, `Dockerfile`, entrypoint, scripts PowerShell, workflow de CI, documentação, arquivos `.env` (inclusive os NÃO versionados presentes no disco) e do histórico completo do Git. Atenção específica a defaults do tipo `${VAR:-valor}` e à ausência de validação de boot que os recuse.
5. Inputs sem tratamento (XSS)	Frontend: varredura de `app/static/*.js` por `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `eval`, `new Function`, `document.write` e `javascript:`; e dos 22 templates por `|safe`, `{% autoescape false %}` e `Markup(`. Backend: busca por HTML montado em Python e por envio de e-mail (não há).

1. Resumo executivo

O MegaSena não tem nenhuma falha crítica, alta ou média. Os 22 handlers de rota foram percorridos um a um: as 8 operações privilegiadas (gestão de contas e configurações, incluindo o `POST /reset` que apaga todo o acervo) exigem o papel `admin` no SERVIDOR, e o template esconde exatamente os mesmos itens — não há um único caso de permissão que só exista no navegador. A varredura de XSS não encontrou nada: zero `|safe` nos 22 templates, zero `innerHTML` nos dois arquivos de JavaScript próprios. O download da planilha de resultados é a superfície mais exposta do sistema e está bem defendida (HTTPS obrigatório, porta 443, recusa de endereço não público, redirecionamento validado a cada salto, teto de 10 MB e proteção contra zip bomb). Os cinco achados são de severidade baixa ou informativa, e nenhum deles é uma porta aberta: são folgas de configuração e decisões de projeto que valem estar registradas.



Achados por severidade e por categoria



2. Pontos fortes e pontos fracos

2.1 O que está protegido (com evidência)

Controle verificado	Evidência no código
Toda rota privilegiada é verificada no servidor As 8 rotas administrativas — as 5 de `/usuarios`, `GET`/`POST /settings` e `POST /reset` — carregam `@admin_required`, que recusa com 403 via `sharedauth.access.requer_papel`. Nenhuma operação privilegiada depende de o botão estar escondido.	`app/web/users.py:58,69,85,124,141` e `app/web/settings.py:29,39,64`
O template esconde exatamente o que a rota recusa O único gate de papel do frontend cobre 'Configurações' e 'Usuários', e ambos correspondem a rotas com `@admin_required`. O próprio comentário no template registra que esconder é apresentação, não controle.	`app/templates/base.html:31-38`
Acesso padrão-nega com lista de PÚBLICOS, não de protegidos `PUBLIC_ENDPOINTS` tem 4 entradas (login, estáticos do Flask e do SharedAuth, health). Uma rota acrescentada amanhã nasce fechada. `prefixo_api=None` e `usar_hx_redirect=True` são coerentes com o app não servir JSON.	`app/__init__.py:47` e `265-272`
Troca de senha obrigatória verificada em toda requisição `requer_troca_de_senha` prende a sessão na tela de troca até ela acontecer, com quatro isenções nomeadas (logout e três estáticos). Verificar só no login deixaria a marca ligada e sem efeito.	`app/__init__.py:284-294`
Sessão amarrada à senha em vigor O `user_loader` separa `id:marca` do cookie e recusa a sessão cuja marca não corresponda mais ao hash guardado — é o que faz trocar a senha derrubar quem entrou com a antiga. Conta desativada também cai já na requisição seguinte.	`app/__init__.py:207-230`
Download da planilha de resultados defendido contra SSRF `normalize_results_source_url` exige HTTPS, recusa credencial embutida e porta diferente de 443 e limita a 200 caracteres; `_ensure_public_host` recusa qualquer endereço resolvido que não seja global (loopback, link-local, RFC1918); os redirecionamentos são seguidos manualmente, no máximo 3, e cada destino passa pelas duas validações de novo.	`app/draws/downloading.py:35-72` e `85-96`
Importação de XLSX com teto de zip bomb Antes de abrir a planilha: no máximo 1.000 membros no arquivo, 100 MB descompactados no total e razão de compressão de 200. Depois: teto de 10.000 linhas e `read_only=True`. `MAX_CONTENT_LENGTH` de 10 MB fecha a entrada, com handler 413 próprio.	`app/draws/importing.py:17-20` e `118-136`
Zero XSS: nenhum `|safe`, nenhum `innerHTML` Varredura completa dos 22 templates por `|safe`, `autoescape` e `Markup(`: nada. Varredura de `app/static/base.js` e `bets.js` por `innerHTML`, `outerHTML`, `insertAdjacentHTML`, `eval`, `new Function`, `document.write` e `javascript:`: nada. Toda a renderização passa pelo autoescape do Jinja.	`app/templates/` (22 arquivos) e `app/static/*.js`
CSP fechada, sem folga de `data:` A política vem do SharedAuth com `imagens_data_uri=False`, e o comentário registra por que a folga anterior deixou de ser necessária (o favicon virou arquivo). As barras do dashboard usam classes `.pct-N` estáticas justamente para não precisar de estilo em runtime.	`app/core/security.py:17-36`
Chave de sessão obrigatória, sem fallback gerado A ausência de `SECRET_KEY` derruba a inicialização com mensagem explícita. Gerar uma chave efêmera mascararia a configuração incompleta e invalidaria toda sessão a cada reinício.	`app/__init__.py:175-181`

Controle verificado	Evidência no código
Limite de login religado corretamente em `view_functions` `aplicar_limite` é chamado depois do registro do blueprint e reatribui a view. Descartar o retorno de `RouteLimit.__call__` deixaria o limite decorado e nunca aplicado — foi o defeito real que este app já teve e que o SharedAuth passou a centralizar.	`app/__init__.py:236-244`
Regra do último administrador com trava serializada no banco `set_active` e `set_role` tomam um advisory lock transacional do PostgreSQL antes de reler o ator e o alvo, então duas requisições concorrentes não conseguem remover o último admin cada uma achando que a outra permanece.	`app/accounts/service.py:202-208 e 152-190`
Mensagem de login única para os três casos de recusa Usuário inexistente, conta inativa e senha errada devolvem o mesmo texto e o mesmo 401 — não há como enumerar contas pela resposta.	`app/web/auth.py:60-67`
Sem segredo no histórico do Git Nenhum arquivo `.env`, `.pem`, `.key` ou de segredo jamais foi adicionado ao repositório, e a varredura de conteúdo por padrão de atribuição de segredo em todos os commits não encontrou nada. O `.gitignore` recusa `.env.*` e o PAT do build entra por BuildKit secret, sem virar camada da imagem.	`git log --all e Dockerfile:57-60`

2.2 Os riscos centrais

• Há cópias redundantes dos mesmos segredos em texto claro: `.env.docker` guarda `POSTGRES_PASSWORD` e `SECRET_KEY` com os mesmos valores que já estão em `.secrets/`, e `create_app` ainda aceita `SECRET_KEY` do ambiente como último recurso. Não houve exposição — o Git e a sincronização de nuvem já recusavam os dois caminhos —, mas cada cópia extra é uma chance a mais.

• As proteções de transporte não moram na aplicação. `MEGA_SENA_FORCE_HTTPS` e `MEGA_SENA_TRUST_PROXY_HEADERS` são `false` por padrão no Compose, o app não redireciona HTTP→HTTPS e não emite HSTS: os três dependem de o Nginx do VPS estar correto e de `.env.vps` ter sido copiado. Nenhum teste cobre essa combinação.

• Só o login tem limite de taxa. Não há `default_limits` global, então rotas de custo alto — importação por link, importação de planilha e o relatório combinatório — não têm teto por requisição.

• O acervo é comum por decisão de projeto, e essa decisão tem consequência prática: qualquer conta com o perfil `operador` apaga aposta salva de qualquer outro, importa concursos e altera o acervo compartilhado. Está documentado, é coerente com o uso individual do mantenedor, e deixa de ser adequado no dia em que uma segunda pessoa passar a usar o sistema.

2.3 Categorias que não se aplicam a esta stack

• **1. Banco sem tranca (isolamento de inquilino/dono)** — Não há inquilino nem dono a isolar. `Draw`, `GeneratedBet` e `Config` não têm coluna de proprietário em nenhuma das 5 revisões Alembic, e o AGENTS.md declara a escolha por escrito: 'o sistema autentica, mas não particiona dados: qualquer conta autenticada acessa o acervo comum'. Não há, portanto, filtro de dono ausente — não existe filtro de dono a ter. A consequência prática dessa escolha está registrada como MS-04, para que ela seja uma decisão visível e não uma omissão.

• **3. IDOR (objeto de outro dono)** — Pela mesma razão: sem dono, não há 'objeto de outro dono' a alcançar. Os 4 handlers que recebem ID no caminho foram percorridos assim mesmo. As 3 rotas de `/usuarios/<id>/...` exigem `admin` e verificam o alvo (`_get_user_or_404`, e a recusa de desativar a própria conta); `POST /bets/saved/<id>/delete` alcança qualquer aposta salva por qualquer conta autenticada — o que é o acervo comum funcionando como projetado, e está em MS-04.

• **5. XSS no backend (e-mail/template)** — A aplicação não envia e-mail e não monta HTML em Python: toda saída passa por `render_template` com o autoescape do Jinja ligado.

3. Achados detalhados

Um bloco por categoria. Todo achado abaixo foi conferido no código real: arquivo, linha e trecho vêm da árvore auditada, não de inferência.

1. Isolamento de dados

Severidade	Arquivo:linha	Descrição

BAIXA	`app/draws/downloading.py:60-72 e 85-96`	**[MS-05] Janela TOCTOU entre a validação de IP público e a conexão HTTP**
		`_ensure_public_host` resolve o nome com `socket.getaddrinfo` e recusa se algum endereço não for global. Logo em seguida, `opener.open(request)` resolve o nome DE NOVO, por conta própria, e conecta ao que vier dessa segunda resolução. As duas resoluções são independentes.
		Por que é explorável: Quem controla o servidor DNS do domínio configurado pode responder um endereço público na primeira consulta e um endereço interno (127.0.0.1, a rede do Compose, o metadata service da nuvem) na segunda — o padrão conhecido como DNS rebinding. A validação passa e a conexão vai para outro lugar.
		Impacto: Requisição HTTPS emitida de dentro da rede da aplicação para um destino interno escolhido pelo atacante. A extração de dado é limitada: a resposta só é aceita com `Content-Type` de XLSX, o corpo para em 10 MB e o conteúdo entra no importador de planilha, que recusa o que não for uma planilha bem-formada. O ganho real fica em sondagem de rede pelos tempos e pelas mensagens de erro.
		Condição de explorabilidade: Exige DUAS condições simultâneas: uma conta `admin` (só ela chega a `POST /settings`, o único ponto que grava `results_source_url`) e controle do DNS do domínio informado. Com a URL padrão da Caixa, não é alcançável.
		
		`for _ in range(MAX_REDIRECTS + 1):`
		`_ensure_public_host(current_url) # 1a resolucao: valida`
		`request = Request(current_url, headers={...})`
		`response = opener.open(request, timeout=20) # 2a resolucao: conecta`

Severidade	Arquivo:linha	Descrição
INFORMATIVA	`app/web/bets.py:483-510 + app/web/contests.py:70,99`	**[MS-04] Acervo comum: qualquer `operador` apaga aposta de qualquer um e altera os concursos** `POST /bets/saved/<int:bet_id>/delete` chama `delete_saved_bet(bet_id)` sem nenhuma verificação de papel e sem noção de dono — não há dono a verificar. Do mesmo modo, `POST /contests/import` e `POST /contests/import-link` gravam no acervo compartilhado de concursos sem exigir `admin`. **Por que é explorável:** É a decisão de projeto funcionando como escrita, não um descuido: o AGENTS.md declara que o sistema não particiona dados, e o schema confirma (nenhuma tabela de domínio tem coluna de proprietário). Registrado aqui porque é exatamente o que faz as categorias 1 e 3 não se aplicarem, e porque o efeito é real: uma conta `operador` — o papel que toda conta nova recebe — apaga o trabalho de outra pessoa e reescreve o acervo. **Impacto:** Nenhum hoje, no uso individual do mantenedor. Deixa de ser adequado no dia em que uma segunda pessoa usar o sistema com conta própria, e nesse dia a mudança é de schema (coluna de dono nas apostas) e não de rota. **Condição de explorabilidade:** Todas as três rotas exigem sessão. A ação verdadeiramente destrutiva — `POST /reset`, que apaga TODOS os concursos e apostas — essa sim exige `admin` (app/web/settings.py:63-64) e é auditada.

Severidade	Arquivo:linha	Descrição

		`@bp.post("/bets/saved/<int:bet_id>/delete")`
		`def delete_bet(bet_id: int):`
		`try:`
		`deleted = delete_saved_bet(bet_id)`

2. Permissão definida no navegador

Severidade	Arquivo:linha	Descrição

BAIXA	`app/__init__.py:199 e 244`	**[MS-03] Só o login tem limite de taxa; não há teto global e as rotas caras ficam sem nenhum**
		`iniciar_limiter(app)` é chamado sem `limites_padrao`, então o Flask-Limiter fica sem `default_limits`. A única rota com limite é `web.login`, por `aplicar_limite(..., LIMITE_LOGIN_PADRAO)`. As 21 rotas restantes não têm teto nenhum.
		Por que é explorável: Três rotas fazem trabalho caro por requisição e estão abertas a qualquer conta autenticada: `POST /contests/import-link` dispara um download externo de até 10 MB com timeout de 20 s por tentativa e até 3 redirecionamentos; `POST /contests/import` processa uma planilha de até 10.000 linhas; e `GET /rationale` monta o relatório combinatório. Sem teto, uma sessão válida repete qualquer uma delas em laço e ocupa os workers do gunicorn.
		Impacto: Indisponibilidade do serviço a partir de uma única conta autenticada, sem precisar de privilégio. É negação de serviço, não vazamento — por isso a severidade baixa.
		Condição de explorabilidade: Exige uma sessão válida. O sistema é de uso individual e as contas são poucas e provisionadas por administrador, o que reduz muito a probabilidade — mas não muda a ausência do controle.
		
		`limiter = iniciar_limiter(app)`
		`...`
		`aplicar_limite(app, limiter, "web.login", LIMITE_LOGIN_PADRAO)`

4. Chaves expostas

Severidade	Arquivo:linha	Descrição
BAIXA	`.env.docker:3,6 (não versionado) + app/__init__.py:171-181`	**[MS-01] Segredos duplicados em texto claro em `.env.docker`, e o fallback de ambiente que os mantém vivos** O arquivo `.env.docker` guarda `POSTGRES_PASSWORD=<redigido>` e `SECRET_KEY=<redigido>` em texto claro — os MESMOS valores que já estão em `.secrets/postgres_password.txt` e `.secrets/secret_key.txt`, confirmado por comparação de hash. São cópias redundantes, sobra do fluxo anterior ao Docker secret. **Por que é explorável:** Não é exposição, e vale registrar o que foi verificado: `.gitignore:35` recusa `.env.*`; o histórico completo do Git não tem nenhum arquivo de segredo; e `.dropboxignore:34-35` já excluía `.env` e `.env.docker` da sincronização de nuvem ANTES desta auditoria (conferido em `git show HEAD:.dropboxignore`). A pasta `.secrets/` foi verificada em dropbox.com, inclusive na lixeira: nunca subiu. O que resta é superfície — o mesmo segredo em dois arquivos em vez de um — e o fato de a cadeia de resolução da chave em `create_app` terminar em `os.environ.get("SECRET_KEY")` (linha 176), com o comentário ao lado registrando que a forma existe 'para execução manual antiga'. Enquanto esse caminho continuar aceito e em silêncio, a cópia do `.env.docker` não é sobra inerte: é uma segunda fonte de verdade para a chave que assina as sessões.

Severidade	Arquivo:linha	Descrição

		Impacto: Nenhum comprometimento conhecido. O custo é de higiene e de raio de alcance: cada cópia adicional de um segredo é mais um lugar de onde ele pode escapar por um caminho não previsto — um arquivo compactado, um `docker compose --env-file` numa invocação manual, um compartilhamento de pasta.
		Condição de explorabilidade: O `compose.yaml` operacional NÃO consome essas duas variáveis: concede os dois segredos por arquivo Docker secret (`DB_PASSWORD_FILE` e `SECRET_KEY_FILE`, linhas 83-84), montados de `.secrets/`. A cópia do `.env.docker` só entra em jogo numa invocação manual que carregue o arquivo no ambiente.
		
		`POSTGRES_PASSWORD=<redigido>`
		`SECRET_KEY=<redigido>`

BAIXA	`compose.yaml:89-90 + app/__init__.py:114,129-139`	**[MS-02] Proteções de transporte só existem fora da aplicação, e o padrão do Compose é o inseguro**
		`MEGA_SENA_FORCE_HTTPS` e `MEGA_SENA_TRUST_PROXY_HEADERS` valem `false` quando não informados. A primeira é usada apenas para decidir o atributo `Secure` dos cookies (`configurar_sessao(https_obrigatorio=force_https)`); a aplicação não redireciona HTTP→HTTPS e não emite `Strict-Transport-Security` em nenhum ponto. A segunda decide se o `ProxyFix` é instalado — e sem ele o limitador chaveia pelo IP do gateway do Docker, igual para todos.
		Por que é explorável: As três garantias — cookie `Secure`, HSTS e chaveamento correto do rate limit — dependem de duas variáveis de ambiente e de uma diretiva de Nginx que vive fora do repositório do serviço. Uma recriação do VPS que esqueça de copiar `.env.vps` sobe o serviço em silêncio: o cookie de sessão passa a viajar sem `Secure` e o limite de 10 tentativas por minuto vira um balde único somando o mundo inteiro. Nada falha, nada avisa, e o sintoma só aparece se alguém for medir.
		Impacto: Numa implantação com essas variáveis ausentes: cookie de sessão interceptável numa requisição HTTP (o Nginx redireciona, mas o cookie já foi enviado se o navegador não conhecer o HSTS) e proteção de força bruta do login sem efeito prático.
		Condição de explorabilidade: A implantação atual está correta — `.env.vps.example` documenta `MEGA_SENA_FORCE_HTTPS=true` e `MEGA_SENA_TRUST_PROXY_HEADERS=true`, e o HSTS existe em `deploy/nginx/megasena.conf:26`. O achado é sobre a ausência de qualquer trava que impeça a configuração insegura de subir.
		
		`MEGA_SENA_TRUST_PROXY_HEADERS:`
		`${MEGA_SENA_TRUST_PROXY_HEADERS:-false}`
		`MEGA_SENA_FORCE_HTTPS: ${MEGA_SENA_FORCE_HTTPS:-false}`

4. Recomendações priorizadas

Prio	Ação	Achados
P1	Fazer a aplicação recusar-se a subir quando `MEGA_SENA_TRUSTED_HOSTS` apontar para um domínio público e `MEGA_SENA_FORCE_HTTPS` estiver desligado. É a checagem que transforma a configuração insegura de silenciosa em impossível, e cabe em poucas linhas de `create_app`.	MS-02
P2	Remover `POSTGRES_PASSWORD` e `SECRET_KEY` de `.env.docker` — `scripts/provision_secrets.ps1` já tem `-RemoveLegacyValues` exatamente para isso — e registrar aviso de nível WARNING quando a `SECRET_KEY` vier do ambiente em vez do arquivo, para o caminho de compatibilidade deixar de ser silencioso. Rotacionar não é necessário: os valores não vazaram.	MS-01
P2	Definir `limites_padrao` global em `iniciar_limiter` (por exemplo `['300 per hour', '60 per minute']`) e um limite dedicado, mais estreito, para as duas rotas de importação e para `/rationale`. Usar `aplicar_limite` com `override_defaults=True`, como o ConfortoTermico já faz nas rotas de polling.	MS-03
P3	Fechar a janela de DNS rebinding resolvendo o host UMA vez e conectando ao endereço já validado, ou restringir `results_source_url` a uma lista curta de domínios aceitos — o valor real muda com pouquíssima frequência.	MS-05
P3	Registrar num ADR a decisão de acervo comum e o gatilho para revisá-la ('quando uma segunda pessoa passar a usar o sistema com conta própria').	MS-04

5. Issues para o GitHub

Cada bloco abaixo é o texto COMPLETO de uma issue, em Markdown, pronto para copiar e colar. Achados triviais do mesmo tema foram agrupados numa issue só, para não gerar ruído no rastreador.

Issue 1 — [Segurança] Segredos duplicados em `.env.docker` e fallback silencioso de `SECRET_KEY`

````
--- ISSUE 1 ---
Título: [Segurança] Segredos duplicados em `.env.docker` e fallback silencioso de `SECRET_KEY`
Labels: security, severity:baixa, segredos, higiene

## Problema

`.env.docker` guarda `POSTGRES_PASSWORD` e `SECRET_KEY` em texto claro — os
MESMOS valores que já estão em `.secrets/postgres_password.txt` e
`.secrets/secret_key.txt`, confirmado por comparação de hash. São cópias
redundantes, sobra do fluxo anterior ao Docker secret.

### O que NAO e

Não houve exposição. O que foi verificado:

- `.gitignore:35` recusa `.env.*`, confirmado por `git check-ignore`;
- o histórico completo do Git não tem nenhum arquivo de segredo;
- `.dropboxignore:34-35` ja excluia `.env` e `.env.docker` da sincronizacao de
  nuvem ANTES desta auditoria — conferido em `git show HEAD:.dropboxignore`;
- a pasta `.secrets/` foi verificada em dropbox.com, inclusive na lixeira:
  nunca subiu.

### Por que ainda vale corrigir

Superfície. O mesmo segredo existe em dois arquivos em vez de um, e a cadeia de
resolução da chave termina em `os.environ.get("SECRET_KEY")`, com o comentário
ao lado registrando que a forma existe "para execução manual antiga". Enquanto
esse caminho continuar aceito **e em silêncio**, a cópia do `.env.docker` não é
sobra inerte: é uma segunda fonte de verdade para a chave que assina as sessões.

## Evidência

`app/__init__.py:171-181`

```python
configured_secret = str(app.config.get("SECRET_KEY") or "").strip()
secret_key = configured_secret or _ler_segredo_por_arquivo("SECRET_KEY")
# `SECRET_KEY` no ambiente e compatibilidade para execucao manual antiga;
# o Compose concede a chave exclusivamente por arquivo Docker secret.
if not secret_key:
    secret_key = os.environ.get("SECRET_KEY", "").strip()
```

`compose.yaml:83-84` — o caminho que de fato vale em operação:

```yaml
      DB_PASSWORD_FILE: /run/secrets/postgres_password
      SECRET_KEY_FILE: /run/secrets/secret_key
```

## Impacto

Nenhum comprometimento conhecido. O custo é de higiene e de raio de alcance:
cada cópia adicional de um segredo é mais um lugar de onde ele pode escapar por
um caminho não previsto — um arquivo compactado, um `docker compose --env-file`
manual, um compartilhamento de pasta.

## Sugestão de correção

O script já sabe fazer a primeira metade, e recusa agir se algum arquivo de
`.secrets/` estiver ausente ou vazio (linhas 78-83):

```powershell
.\scripts\provision_secrets.ps1 -RemoveLegacyValues
```

A segunda metade é tornar o fallback audível, em `create_app`:

```python
if not secret_key:
    secret_key = os.environ.get("SECRET_KEY", "").strip()
    if secret_key:
        _log.warning(
            "SECRET_KEY veio do ambiente, nao de SECRET_KEY_FILE. "
            "E o caminho de compatibilidade; o Compose concede por arquivo."
        )
````
````

````
```

Rotacionar não é necessário: os valores não vazaram.

## Critérios de aceite

- [ ] `.env.docker` não contém mais `POSTGRES_PASSWORD` nem `SECRET_KEY`
- [ ] A pilha sobe normalmente com os segredos vindos de `.secrets/`
- [ ] Subir com `SECRET_KEY` no ambiente registra WARNING identificável no log
- [ ] Subir com `SECRET_KEY_FILE` não registra esse WARNING
- [ ] Teste cobrindo os dois casos acima
- [ ] `.dropboxignore` mantém `.env*`, `.secrets/` e `.certs/` (regressão)
--- FIM ISSUE 1 ---
````

Issue 2 — [Segurança] Configuração insegura de transporte sobe em silêncio

````
--- ISSUE 2 ---
Título: [Segurança] Configuração insegura de transporte sobe em silêncio
Labels: security, severity:baixa, implantacao

## Problema

As proteções de transporte deste serviço moram todas fora da aplicação, e o
padrão do Compose é o inseguro:

- `MEGA_SENA_FORCE_HTTPS` vale `false` quando não informado — e é o único
  interruptor que liga `Secure` nos cookies de sessão;
- `MEGA_SENA_TRUST_PROXY_HEADERS` vale `false` quando não informado — sem ele
  não há `ProxyFix`, e o limitador de taxa chaveia pelo IP do gateway do
  Docker, igual para todas as requisições;
- a aplicação não redireciona HTTP→HTTPS e não emite
  `Strict-Transport-Security` em ponto nenhum: os dois existem apenas em
  `deploy/nginx/megasena.conf:26`.

### Por que é explorável

Uma recriação do VPS que esqueça de copiar `.env.vps` sobe o serviço sem que
nada falhe: o cookie de sessão passa a viajar sem `Secure`, e o limite de 10
tentativas de login por minuto vira um balde único somando o mundo inteiro — o
adivinhador divide o orçamento com os usuários legítimos em vez de ter o seu
próprio. Nenhum teste cobre a combinação, e o sintoma só aparece se alguém for
medir.

## Evidência

`compose.yaml:89-90`

```yaml
MEGA_SENA_TRUST_PROXY_HEADERS: ${MEGA_SENA_TRUST_PROXY_HEADERS:-false}
MEGA_SENA_FORCE_HTTPS: ${MEGA_SENA_FORCE_HTTPS:-false}
```

`app/__init__.py:114` e `129-139`

```python
force_https = ler_flag("MEGA_SENA_FORCE_HTTPS")
...
configurar_sessao(app, ..., https_obrigatorio=force_https, ...)
if ler_flag("MEGA_SENA_TRUST_PROXY_HEADERS"):
    app.wsgi_app = ProxyFix(app.wsgi_app, x_for=1, x_host=1, x_proto=1)
```

## Impacto

Numa implantação com essas variáveis ausentes: cookie de sessão interceptável
numa requisição HTTP e proteção de força bruta do login sem efeito prático.

A implantação atual está correta (`.env.vps.example` documenta as duas em
`true`), mas isso é uma propriedade do servidor, não do repositório.

## Sugestão de correção

Fazer a configuração insegura falhar ao subir, em vez de subir calada. Em
`create_app`, depois de resolver `TRUSTED_HOSTS`:

```python
_LOOPBACK = {"localhost", "127.0.0.1", "[::1]"}
hosts = app.config["TRUSTED_HOSTS"]
if any(h not in _LOOPBACK for h in hosts) and not force_https:
    raise RuntimeError(
        "MEGA_SENA_TRUSTED_HOSTS aponta para host publico com "
        "MEGA_SENA_FORCE_HTTPS desligado: o cookie de sessao sairia sem Secure."
    )
```

A mesma checagem pode exigir `MEGA_SENA_TRUST_PROXY_HEADERS` quando houver host
público, já que nessa topologia sempre há um proxy à frente.
````
````

````
## Critérios de aceite

- [ ] Subir com host público e `MEGA_SENA_FORCE_HTTPS=false` levanta `RuntimeError` com mensagem
     que nomeia as duas variáveis
- [ ] Subir com hosts de loopback e `FORCE_HTTPS=false` continua funcionando (desenvolvimento
    local em HTTP)
- [ ] Subir com host público e as duas variáveis em `true` funciona
- [ ] Teste cobrindo os três casos acima, no estilo de `tests/test_proxy_configuration.py`
- [ ] `docs/deployment-vps.md` registra que a checagem existe e o que fazer quando ela disparar
--- FIM ISSUE 2 ---
````

Issue 3 — [Segurança] Sem limite de taxa global: rotas de importação e relatório ficam sem teto

````
--- ISSUE 3 ---
Título: [Segurança] Sem limite de taxa global: rotas de importação e relatório ficam sem teto
Labels: security, severity:baixa, rate-limit

## Problema

`iniciar_limiter(app)` é chamado sem `limites_padrao`, então o Flask-Limiter
fica sem `default_limits`. A única rota com limite em todo o aplicativo é
`web.login`. As 21 rotas restantes não têm teto nenhum.

### Por que é explorável

Três rotas fazem trabalho caro por requisição e estão abertas a qualquer conta
autenticada, sem exigir `admin`:

- `POST /contests/import-link` — dispara um download externo de até 10 MB, com
  timeout de 20 s por tentativa e até 3 redirecionamentos seguidos;
- `POST /contests/import` — processa uma planilha de até 10.000 linhas;
- `GET /rationale` — monta o relatório combinatório.

Uma sessão válida repete qualquer uma delas em laço e ocupa os workers do
gunicorn indefinidamente.

## Evidência

`app/__init__.py:199` e `app/__init__.py:244`

```python
    limiter = iniciar_limiter(app)          # sem limites_padrao
    ...
    aplicar_limite(app, limiter, "web.login", LIMITE_LOGIN_PADRAO)
```

Rotas de custo alto, todas sem decorator de limite:
`app/web/contests.py:70`, `app/web/contests.py:99`, `app/web/bets.py:234`.

## Impacto

Indisponibilidade do serviço a partir de uma única conta autenticada, sem
precisar de privilégio. É negação de serviço, não vazamento.

## Sugestão de correção

Definir um teto global e um teto dedicado, mais estreito, para as rotas caras:

```python
limiter = iniciar_limiter(
    app,
    limites_padrao=["300 per hour", "60 per minute"],
)
...
aplicar_limite(
    app, limiter,
    ("web.import_upload", "web.import_from_link", "web.rationale"),
    "10 per minute; 100 per hour",
    override_defaults=True,
)
```

O padrão de `aplicar_limite` com `override_defaults=True` já é usado no
ConfortoTermico para as rotas de polling e resolve a religação de
`view_functions` corretamente.

## Critérios de aceite

- [ ] `iniciar_limiter` recebe `limites_padrao` e o valor está registrado com o motivo em
    comentário
- [ ] As duas rotas de importação e `/rationale` têm limite dedicado mais estreito que o global
- [ ] Teste: a 11ª chamada seguida a `/contests/import-link` na mesma janela responde 429
- [ ] Teste: o polling normal da tela de concursos não é afetado pelo limite global
- [ ] O limite do login (10/min) continua valendo e o teste existente continua verde
--- FIM ISSUE 3 ---
````

Issue 4 — [Segurança] DNS rebinding contorna a validação de host público do download de resultados

````
--- ISSUE 4 ---
Título: [Segurança] DNS rebinding contorna a validação de host público do download de resultados
Labels: security, severity:baixa, ssrf

## Problema

`fetch_results_xlsx` valida o destino e depois conecta, e as duas coisas
resolvem o nome de forma independente:

1. `_ensure_public_host(current_url)` chama `socket.getaddrinfo` e recusa se
   algum endereço resolvido não for global;
2. `opener.open(request)` resolve o nome DE NOVO, por conta própria, e conecta
   ao que vier dessa segunda resolução.

### Por que é explorável

Quem controla o servidor DNS do domínio configurado responde um endereço
público na primeira consulta e um endereço interno na segunda — `127.0.0.1`, a
rede do Compose, o serviço de metadados da nuvem. A validação passa e a conexão
vai para outro lugar. É o padrão conhecido como DNS rebinding.

## Evidência

`app/draws/downloading.py:85-96`

```python
    for _ in range(MAX_REDIRECTS + 1):
        _ensure_public_host(current_url)          # 1a resolucao: valida
        request = Request(current_url, headers={"User-Agent": "MegaSena/1.0"})
        try:
            response = opener.open(request, timeout=20)   # 2a resolucao: conecta
```

`app/draws/downloading.py:60-72`

```python
def _ensure_public_host(url: str) -> None:
    hostname = urlsplit(url).hostname
    ...
    addresses = socket.getaddrinfo(hostname, 443, type=socket.SOCK_STREAM)
    if not addresses or any(
        not ipaddress.ip_address(address[4][0]).is_global for address in addresses
    ):
        raise ResultsDownloadError("O link da planilha deve apontar para um servidor publico.")
```

## Impacto

Requisição HTTPS emitida de dentro da rede da aplicação para um destino interno
escolhido pelo atacante. A extração de dado é limitada — a resposta só é aceita
com `Content-Type` de XLSX, o corpo para em 10 MB e o conteúdo vai para o
importador de planilha, que recusa o que não for planilha bem-formada. O ganho
real fica em sondagem de rede pelos tempos de resposta e pelas mensagens de
erro.

**Condição de explorabilidade:** exige DUAS coisas ao mesmo tempo — uma conta
`admin` (só ela chega a `POST /settings`, o único ponto que grava
`results_source_url`) e controle do DNS do domínio informado. Com a URL padrão
da Caixa, não é alcançável.

## Sugestão de correção

Resolver o host uma vez e conectar ao endereço já validado, ou fechar o destino
numa lista curta de domínios aceitos — o valor real muda com pouquíssima
frequência:

```python
DOMINIOS_ACEITOS = frozenset({"servicebus3.caixa.gov.br"})

def normalize_results_source_url(value: object) -> str:
    ...
    if parsed.hostname not in DOMINIOS_ACEITOS:
        raise ValueError("O link da planilha deve apontar para a fonte oficial.")
```

A lista fecha a categoria inteira (rebinding incluído) sem exigir que se
reescreva o cliente HTTP, e mantém as validações atuais como defesa em
profundidade.

## Critérios de aceite

- [ ] O destino do download é restringido por lista de domínios OU o endereço validado é o mesmo
     usado na conexão
- [ ] As validações atuais (HTTPS, porta 443, sem credencial embutida, teto de 200 caracteres,
    IP global) continuam valendo
````

```
- [ ] Teste: URL fora da lista/regra é recusada em `POST /settings`, antes de qualquer conexão
- [ ] Teste: a URL padrão da Caixa continua aceita
- [ ] Teste: o limite de 3 redirecionamentos e o teto de 10 MB continuam verdes
- [ ] `docs/business-rules.md` registra a restrição e como alterar a lista
--- FIM ISSUE 4 ---
```

Issue 5 — [Segurança] Registrar em ADR a decisão de acervo comum sem dono

```
--- ISSUE 5 ---
Título: [Segurança] Registrar em ADR a decisão de acervo comum sem dono
Labels: security, severity:informativa, documentacao, adr

## Problema

O sistema autentica mas não particiona dados: nenhuma tabela de domínio tem
coluna de proprietário, e qualquer conta autenticada — inclusive o perfil
`operador`, que é o padrão de toda conta nova — apaga aposta salva de qualquer
outra pessoa e reescreve o acervo de concursos.

Isso é decisão de projeto, está no `AGENTS.md` e foi confirmado no schema. Não é
um defeito. O que falta é o registro da decisão com a alternativa considerada e
o gatilho para revisá-la — o `AGENTS.md` descreve o estado atual, não a escolha.

### Por que importa

É exatamente essa decisão que faz duas das cinco categorias desta auditoria
("banco sem tranca" e "IDOR") não se aplicarem. Uma auditoria futura que não a
encontre registrada vai reabrir a mesma discussão, ou pior: vai tratar a
ausência de filtro de dono como achado.

## Evidência

`app/web/bets.py:483-486`

```python
@bp.post("/bets/saved/<int:bet_id>/delete")
def delete_bet(bet_id: int):
    try:
        deleted = delete_saved_bet(bet_id)
```

`app/web/contests.py:70` e `app/web/contests.py:99` — importação de concursos,
sem `@admin_required`.

Contraste, na mesma base: `app/web/settings.py:63-64` — `POST /reset`, que apaga
TODOS os concursos e apostas, exige `@admin_required` e é auditado.

Impacto

Nenhum hoje, no uso individual do mantenedor. Deixa de ser adequado no dia em
que uma segunda pessoa passar a usar o sistema com conta própria — e nesse dia a
mudança é de schema (coluna de dono em `generated_bets`), não de rota.

Sugestão de correção

Escrever `docs/adr/0002-acervo-comum-sem-dono.md` contendo:

- a decisão: apostas e concursos são acervo comum; papel só separa
  administração de operação;
- a alternativa considerada e recusada: coluna de dono em `generated_bets` com
  filtro por `current_user`;
- por que foi recusada agora: uso individual, e a coluna teria custo de
  migração sem leitor;
- **o gatilho de revisão**: a primeira conta de uma segunda pessoa;
- o plano de migração, em uma frase: revisão Alembic acrescentando
  `owner_id` em `generated_bets`, com backfill para a conta administrativa
  existente, e filtro em `list_recent_generations_with_bets` e
  `delete_saved_bet`.

Critérios de aceite

- [ ] ADR criado em `docs/adr/`, seguindo o formato de `0001-gestao-administrativa-de-contas.md`
- [ ] O ADR nomeia o gatilho de revisão de forma verificável
- [ ] O ADR descreve o plano de migração, incluindo a revisão Alembic necessária
- [ ] `AGENTS.md` referencia o ADR no ponto em que hoje afirma que o sistema não particiona
    dados
- [ ] Nenhuma mudança de comportamento: é documentação
--- FIM ISSUE 5 ---
```